Practical 4 — Building a Transformer from Scratch (PyTorch)

This notebook implements the core Transformer components (input embedding, positional encoding, scaled dot-product attention, multi-head attention, position-wise feed-forward, encoder & decoder layers, and the full encoder-decoder stack) using PyTorch. A small toy example runs a forward pass to verify shapes and gradients.

In [21]:
```python
# Basic imports and device check
import math
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
from typing import Optional

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('device =', device)
```

```
device = cuda
```

In [22]:
```python
# Positional Encoding (batch-first implementation)
class PositionalEncoding(nn.Module):
    def __init__(self, d_model: int, max_len: int = 5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)  # (max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1).float()  # (max_len,1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-(math.log(1000
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)  # (1, max_len, d_model)
        self.register_buffer('pe', pe)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (batch_size, seq_len, d_model)
        x = x + self.pe[:, :x.size(1)].to(x.dtype)
        return x
```

In [23]:
```python
# Scaled Dot-Product Attention
def scaled_dot_product_attention(q, k, v, mask: Optional[torch.Tensor] = None, drop
    # q, k, v: (B*heads, seq_len, head_dim) or (batch, seq_len, head_dim) depending
    d_k = q.size(-1)
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)
    if mask is not None:
        # mask expected to be broadcastable to scores shape; masked positions shoul
        scores = scores.masked_fill(mask == 0, float('-inf'))
    p_attn = F.softmax(scores, dim=-1)
    if dropout is not None:
        p_attn = dropout(p_attn)
    return torch.matmul(p_attn, v), p_attn
```

In [24]:
```python
# Multi-Head Attention (batch-first)
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model: int, num_heads: int, dropout: float = 0.1):
        super().__init__()
        assert d_model % num_heads == 0, 'd_model must be divisible by num_heads'
        self.d_model = d_model
        self.h = num_heads
        self.d_k = d_model // num_heads
        # three projection matrices for q,k,v and one for output
        self.linears = nn.ModuleList([nn.Linear(d_model, d_model) for _ in range(4)
        self.attn = None
        self.dropout = nn.Dropout(p=dropout)

    def _expand_mask(self, mask, batch_size, seq_len_q, seq_len_k):
        if mask is None:
            return None
        # mask can be (batch, seq_k) or (batch, seq_q, seq_k) or (batch, 1, 1, seq_
        if mask.dim() == 2:
            mask = mask.unsqueeze(1).unsqueeze(1)  # (batch,1,1,seq_k)
        elif mask.dim() == 3:
            mask = mask.unsqueeze(1)  # (batch,1,seq_q,seq_k)
        # now (batch,1,seq_q,seq_k); repeat heads
        mask = mask.repeat(1, self.h, 1, 1)  # (batch,heads,seq_q,seq_k)
        return mask.view(batch_size * self.h, mask.size(2), mask.size(3))

    def forward(self, query, key, value, mask: Optional[torch.Tensor] = None):
        # query/key/value: (batch, seq, d_model)
        batch_size = query.size(0)
        seq_len_q = query.size(1)
        seq_len_k = key.size(1)
        # linear projections
        q = self.linears[0](query)
        k = self.linears[1](key)
        v = self.linears[2](value)
        # split heads: (batch, heads, seq_len, d_k)
        q = q.view(batch_size, -1, self.h, self.d_k).transpose(1, 2)
        k = k.view(batch_size, -1, self.h, self.d_k).transpose(1, 2)
        v = v.view(batch_size, -1, self.h, self.d_k).transpose(1, 2)
        # merge batch and heads for efficient attention computation: (batch*heads,
        q = q.contiguous().view(batch_size * self.h, -1, self.d_k)
        k = k.contiguous().view(batch_size * self.h, -1, self.d_k)
        v = v.contiguous().view(batch_size * self.h, -1, self.d_k)
        # expand mask to match (batch*heads, seq_q, seq_k)
        attn_mask = self._expand_mask(mask, batch_size, seq_len_q, seq_len_k) if ma
        x, attn = scaled_dot_product_attention(q, k, v, mask=attn_mask, dropout=sel
        # x: (batch*heads, seq_len_q, d_k) -> (batch, heads, seq_len_q, d_k)
        x = x.view(batch_size, self.h, -1, self.d_k).transpose(1, 2).contiguous()
        x = x.view(batch_size, -1, self.h * self.d_k)  # (batch, seq_len_q, d_model
        return self.linears[3](x)  # final linear projection
```

Position-wise Feed-Forward Network

A two-layer feed-forward network applied independently to each position. This increases the model's capacity and adds non-linearity between attention layers.

In [25]:
```
class PositionwiseFeedForward(nn.Module):
    def __init__(self, d_model, d_ff, dropout=0.1):
        super().__init__()
        self.w_1 = nn.Linear(d_model, d_ff)
        self.w_2 = nn.Linear(d_ff, d_model)
        self.dropout = nn.Dropout(dropout)
    def forward(self, x):
        return self.w_2(self.dropout(F.relu(self.w_1(x))))
```

Encoder Layer

Each encoder layer contains multi-head self-attention followed by a position-wise feed-forward network, with residual connections and layer normalization.

In [26]:
```
class EncoderLayer(nn.Module):
    def __init__(self, d_model, heads, d_ff, dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, heads, dropout)
        self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)
    def forward(self, x, mask=None):
        x2 = self.self_attn(x, x, x, mask=mask)
        x = x + self.dropout(x2)
        x = self.norm1(x)
        x2 = self.feed_forward(x)
        x = x + self.dropout(x2)
        return self.norm2(x)
```

Decoder Layer

The decoder layer has masked self-attention (preventing access to future tokens), encoder-decoder attention, and a feed-forward network. Residual connections and layer normalization are used throughout.

In [27]:
```
class DecoderLayer(nn.Module):
    def __init__(self, d_model, heads, d_ff, dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, heads, dropout)
        self.src_attn = MultiHeadAttention(d_model, heads, dropout)
        self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)
    def forward(self, x, memory, src_mask=None, tgt_mask=None):
```

```
        x2 = self.self_attn(x, x, x, mask=tgt_mask)
        x = x + self.dropout(x2)
        x = self.norm1(x)
        x2 = self.src_attn(x, memory, memory, mask=src_mask)
        x = x + self.dropout(x2)
        x = self.norm2(x)
        x2 = self.feed_forward(x)
        x = x + self.dropout(x2)
        return self.norm3(x)
```

Encoder & Decoder Stacks

The encoder and decoder are stacks of identical layers. A final layer normalization is applied after the stack.

In [28]:
```
class Encoder(nn.Module):
    def __init__(self, d_model, N, heads, d_ff, dropout=0.1):
        super().__init__()
        self.layers = nn.ModuleList([EncoderLayer(d_model, heads, d_ff, dropout) fo
        self.norm = nn.LayerNorm(d_model)
    def forward(self, x, mask=None):
        for layer in self.layers:
            x = layer(x, mask=mask)
        return self.norm(x)

class Decoder(nn.Module):
    def __init__(self, d_model, N, heads, d_ff, dropout=0.1):
        super().__init__()
        self.layers = nn.ModuleList([DecoderLayer(d_model, heads, d_ff, dropout) fo
        self.norm = nn.LayerNorm(d_model)
    def forward(self, x, memory, src_mask=None, tgt_mask=None):
        for layer in self.layers:
            x = layer(x, memory, src_mask=src_mask, tgt_mask=tgt_mask)
        return self.norm(x)
```

Full Transformer Model

Combine the embedding + positional encoding, encoder stack, decoder stack, and output projection into a full encoder-decoder Transformer.

In [29]:
```
class Transformer(nn.Module):
    def __init__(self, src_vocab, tgt_vocab, d_model=512, N=6, heads=8, d_ff=2048,
        super().__init__()
        self.src_embed = nn.Sequential(nn.Embedding(src_vocab, d_model), Positional
        self.tgt_embed = nn.Sequential(nn.Embedding(tgt_vocab, d_model), Positional
        self.encoder = Encoder(d_model, N, heads, d_ff, dropout)
        self.decoder = Decoder(d_model, N, heads, d_ff, dropout)
        self.out = nn.Linear(d_model, tgt_vocab)
    def forward(self, src, tgt, src_mask=None, tgt_mask=None):
        src_emb = self.src_embed(src)  # (batch, seq, d_model)
        tgt_emb = self.tgt_embed(tgt)
```

```
        memory = self.encoder(src_emb, mask=src_mask)
        out = self.decoder(tgt_emb, memory, src_mask=src_mask, tgt_mask=tgt_mask)
        return self.out(out)
```

Masks & Utilities

Helper functions to build padding masks and subsequent masks used by attention mechanisms.

In [30]:
```
def make_pad_mask(seq, pad_idx=0):
    # seq: (batch, seq_len) -> returns mask (batch,1,1,seq_len) with True for non-p
    return (seq != pad_idx).unsqueeze(1).unsqueeze(1)

def subsequent_mask(size):
    # returns (1, size, size) mask with True in allowed positions (lower triangle i
    attn_shape = (1, size, size)
    subsequent = np.triu(np.ones(attn_shape), k=1).astype('uint8')
    return torch.from_numpy(1 - subsequent).bool()
```

Task — Toy Copy Task (Demonstration)

Train a small Transformer to copy a random input sequence to the output. This demonstrates encoder-decoder attention and masked decoding in a simple supervised setting.

In [31]:
```
# Small training loop for a copy task
torch.manual_seed(0)
src_vocab = tgt_vocab = 12  # include pad=0 and start=1
pad_idx = 0
start_idx = 1
batch_size = 16
src_len = 6
tgt_len = src_len
d_model = 64
N = 2
heads = 4
d_ff = 128
model = Transformer(src_vocab, tgt_vocab, d_model=d_model, N=N, heads=heads, d_ff=d
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss(ignore_index=pad_idx)

def make_batch(batch_size):
    # generate random sequences (no padding needed for this demo)
    src = torch.randint(2, src_vocab, (batch_size, src_len), dtype=torch.long, devi
    tgt = src.clone()
    # decoder input is start token + tgt[:-1]
    tgt_input = torch.cat([torch.full((batch_size, 1), start_idx, dtype=torch.long,
    return src, tgt_input, tgt

# Training
epochs = 3
iters_per_epoch = 80
```

```python
for epoch in range(epochs):
    model.train()
    total_loss = 0.0
    for it in range(iters_per_epoch):
        src, tgt_input, tgt_labels = make_batch(batch_size)
        src_mask = make_pad_mask(src, pad_idx=pad_idx).to(device)
        tgt_pad_mask = make_pad_mask(tgt_input, pad_idx=pad_idx).to(device)
        sub_mask = subsequent_mask(tgt_input.size(1)).to(device)
        sub_mask = sub_mask.unsqueeze(1)
        tgt_mask = tgt_pad_mask & sub_mask
        logits = model(src, tgt_input, src_mask=src_mask, tgt_mask=tgt_mask)
        logits_flat = logits.view(-1, logits.size(-1))
        tgt_flat = tgt_labels.view(-1)
        loss = criterion(logits_flat, tgt_flat)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f'Epoch {epoch+1}/{epochs} — loss: {total_loss/iters_per_epoch:.4f}')

# Show a sample prediction
model.eval()
with torch.no_grad():
    src, tgt_input, tgt_labels = make_batch(4)
    src_mask = make_pad_mask(src, pad_idx=pad_idx).to(device)
    tgt_pad_mask = make_pad_mask(tgt_input, pad_idx=pad_idx).to(device)
    sub_mask = subsequent_mask(tgt_input.size(1)).to(device).unsqueeze(1)
    tgt_mask = tgt_pad_mask & sub_mask
    logits = model(src, tgt_input, src_mask=src_mask, tgt_mask=tgt_mask)
    preds = logits.argmax(dim=-1).cpu().numpy()
    print('Sample inputs (first 4):')
    print(src.cpu().numpy())
    print('Targets:')
    print(tgt_labels.cpu().numpy())
    print('Predictions:')
    print(preds)
```

```
Epoch 1/3 — loss: 1.7160
Epoch 2/3 — loss: 0.4556
Epoch 3/3 — loss: 0.1306
Sample inputs (first 4):
[[11  9  8  3  9  8]
 [ 9  7 10  7  9  6]
 [ 8 10  8  3  9  3]
 [ 5  6  3  3  4  2]]
Targets:
[[11  9  8  3  9  8]
 [ 9  7 10  7  9  6]
 [ 8 10  8  3  9  3]
 [ 5  6  3  3  4  2]]
Predictions:
[[11  9  8  3  9  8]
 [ 9  7 10  7  9  6]
 [ 8 10  8  3  9  3]
 [ 5  6  3  3  4  2]]
```